REFERENCIAL TEÓRICO E TRABALHOS RELACIONADOS

Influência da especificação de requisitos na segurança e na correção funcional de código gerado por modelos de linguagem

Andre Lucas Francino Castelo Branco

Orientador: Mauro José Araújo de Melo

Engenharia de Software - iCEV

Versão parcial para orientação - 3 de novembro de 2026

2 REFERENCIAL TEÓRICO E TRABALHOS RELACIONADOS

2.1 Requisitos e instruções para geração de código

A especificação de requisitos descreve o comportamento esperado do software e as condições de sua aceitação. O Software Engineering Handbook da NASA relaciona os requisitos às funções, qualidades e restrições do produto, destacando clareza, consistência, verificabilidade e rastreabilidade. Esses atributos estabelecem uma ligação entre a descrição do problema e os procedimentos utilizados para avaliar a implementação (NASA, s.d.).

Na geração por modelos de linguagem, a tarefa é apresentada por uma instrução, ou prompt. Neste estudo, a especificação corresponde ao conteúdo dessa instrução sobre comportamento funcional e restrições de segurança. O detalhamento pode definir entradas, respostas e situações de erro, reduzindo decisões deixadas ao gerador. Seu efeito sobre o código, contudo, precisa ser observado: uma descrição mais extensa também pode conter ambiguidades ou exigências conflitantes.

Detalhamento funcional e explicitação de segurança são aspectos distintos da instrução. Definir os filtros de uma consulta não determina como os valores serão incorporados ao acesso ao banco. Da mesma forma, descrever os campos de um pedido não exige, por si só, verificar a autorização de quem o solicita. A distinção permite comparar uma base funcional detalhada com a mesma base acrescida de requisitos de proteção.

Essa separação não trata segurança exclusivamente como requisito não funcional. Autenticar uma identidade e conferir sua autorização também são comportamentos da aplicação. No experimento, o critério de classificação é o conteúdo da instrução: são distinguidas as exigências relativas à função principal e aquelas destinadas a impedir o uso indevido.

2.2 Fraquezas e requisitos de segurança

A Common Weakness Enumeration (CWE) reúne categorias de fraquezas de software e hardware desenvolvidas pela comunidade e mantidas no programa coordenado pela MITRE. Uma categoria identifica um tipo de condição que pode contribuir para uma vulnerabilidade. Ela permite agrupar ocorrências de ferramentas distintas, mas sua atribuição a um trecho de código depende da análise da ocorrência (MITRE, s.d.).

O Application Security Verification Standard (ASVS), da OWASP, apresenta requisitos para o desenvolvimento seguro e a verificação de controles de aplicações. A CWE auxilia a classificar o que pode falhar; o ASVS oferece uma referência para especificar o que o sistema deve atender. O trabalho utiliza como referência a versão 5.0.0 do ASVS (OWASP, 2025).

Uma orientação como “seguir boas práticas” não define quais controles serão exigidos. Já um requisito que condiciona o retorno de um pedido à correspondência entre seu proprietário e o usuário autenticado estabelece uma regra observável. A diferença entre orientação genérica e requisito verificável fundamenta os contrastes da pesquisa. Os requisitos do piloto foram redigidos com referência a práticas de segurança de aplicações; sua correspondência integral com os identificadores do ASVS ainda será formalizada no protocolo principal.

2.3 Correção funcional e critérios de teste

A correção funcional é examinada pelo atendimento aos comportamentos definidos para a tarefa. Neste estudo, essa propriedade é medida por testes de aceitação: requisições são submetidas à aplicação e as respostas são confrontadas com valores esperados. A aprovação indica atendimento aos casos examinados, sem constituir uma prova de correção para todas as entradas possíveis.

O critério que determina o resultado esperado é denominado oráculo de teste. Para uma consulta de produtos, por exemplo, a lista pode ser calculada a partir de registros conhecidos e dos filtros fornecidos. Usar a própria resposta do programa para definir a expectativa tornaria a verificação circular. Por isso, a origem dos dados e as regras de cálculo das expectativas integram o instrumento de avaliação.

Testes funcionais e de segurança podem utilizar a mesma interface e observar propriedades diferentes. O retorno de um pedido ao seu proprietário verifica uma capacidade funcional; o retorno desse pedido a outro usuário evidencia uma falha de proteção. Ambos os casos são necessários: rejeitar todas as requisições impede o acesso indevido, mas também pode tornar a função inutilizável. A análise conjunta depende, assim, de controles de uso legítimo e de solicitações adversas.

2.4 Análise estática e verificação comportamental

A análise estática examina o código sem executar cada cenário de uso. O Bandit constrói uma árvore sintática a partir dos arquivos Python e aplica verificadores a seus nós para identificar problemas recorrentes de segurança (Bandit, s.d.). O Semgrep utiliza regras com correspondência de padrões e análise de fluxo de dados. As regras disponíveis ou definidas pelo usuário determinam parte do que será procurado no código (Semgrep, s.d.).

Um alerta estático representa uma ocorrência para análise. Quando não corresponde à fraqueza indicada, constitui um falso positivo; quando uma fraqueza existente não é sinalizada, há um falso negativo. A concordância entre ferramentas não substitui um critério de referência para confirmar as ocorrências. A ausência de alertas, por sua vez, descreve o resultado da configuração utilizada, e não a segurança completa do programa.

A verificação comportamental observa a aplicação em execução sob condições controladas. Em um teste de autorização, são definidas a identidade solicitante, a propriedade do recurso e a resposta esperada. O acesso legítimo funciona como controle positivo: se o proprietário não consegue consultar o próprio pedido, a rejeição de outro usuário pode decorrer de uma falha funcional. Nesse caso, a avaliação da proteção fica inconclusiva.

Os resultados de segurança dependem, portanto, da propriedade, das entradas e dos instrumentos empregados. Uma política de bloqueio pode ser testada em um número definido de tentativas, sem que esse procedimento cubra todos os limites possíveis. Do mesmo modo, a análise estática pode identificar certos padrões de autorização quando suas regras e informações forem suficientes. O uso complementar dos instrumentos amplia a observação, mas conserva os limites de cada teste.

2.5 Trabalhos relacionados

Os estudos a seguir foram selecionados por sua relação com a segurança do código gerado, a formulação das instruções ou a avaliação conjunta de segurança e funcionalidade. A seleção é narrativa; não se apresenta como revisão sistemática da literatura.

Pearce et al. (2022) avaliaram 1.689 programas produzidos pelo GitHub Copilot em 89 cenários e identificaram vulnerabilidades em aproximadamente 40% deles. O estudo fornece evidência inicial de variação da segurança em condições de geração. Seu resultado corresponde à ferramenta, às tarefas e aos critérios daquele experimento, sem representar uma taxa de falha dos modelos atuais. Perry et al. (2023), em um estudo com participantes, observaram que usuários com o assistente avaliado produziram código menos seguro e demonstraram maior confiança em sua segurança. A unidade de observação humana distingue esse trabalho da comparação direta de gerações realizada aqui.

Khoury et al. (2023) investigaram a segurança de programas gerados pelo ChatGPT e sua resposta a novas solicitações de proteção. Os resultados indicam que o modelo pode reconhecer fraquezas e ainda produzir implementações suscetíveis a ataques. O estudo motiva examinar o conteúdo das instruções, mas o reconhecimento de uma falha não demonstra que sua causa seja apenas a ausência de um requisito explícito.

Tony et al. (2025) combinaram revisão de técnicas de prompt e avaliação de um subconjunto em GPT-3, GPT-3.5 e GPT-4, utilizando 150 instruções do conjunto LLMSecEval. Observaram redução de fraquezas, com destaque para a técnica de crítica e melhoria recursiva. A avaliação também incluiu inspeção manual da adequação à tarefa e da completude; respostas consideradas inválidas puderam ser regeneradas e, depois, descartadas. Portanto, o trabalho não ignora completamente a funcionalidade. Difere deste estudo pelo uso de revisão iterativa em certas condições e pelo tratamento das respostas inválidas, enquanto aqui os impedimentos de execução permanecem visíveis nos denominadores.

Kharma et al. (2026) compararam quatro métodos de prompt em cinco modelos e quatro linguagens, combinando análise estática e revisão manual. Não encontraram redução estatisticamente significativa da frequência ou da densidade de vulnerabilidades entre os métodos, embora tenham observado mudanças nas categorias CWE. O resultado difere do relatado por Tony et al. (2025), mas as técnicas, os modelos e os critérios também diferem. Essa comparação sustenta a investigação das condições de instrução sem pressupor um benefício uniforme da orientação de segurança.

A revisão sistemática de Basic e Giaretta (2026), com 102 estudos, abrange geração, detecção e correção de vulnerabilidades por modelos de linguagem. Seu escopo permite situar diferentes usos dos modelos, que não devem ser tratados como avaliações equivalentes. Detectar uma falha já existente ou reparar código após receber informações sobre ela envolve condições distintas da geração inicial avaliada neste trabalho.

Wang et al. (2026) apresentam o RealSec-bench, composto de tarefas extraídas de repositórios Java. A avaliação distingue correção funcional, segurança e atendimento conjunto, utilizando testes funcionais, análise estática e revisão de alertas por modelos. O estudo compara geração de referência, recuperação de contexto e inclusão de orientações de segurança. Sua avaliação em contexto de repositório difere dos endpoints Python/FastAPI aqui solicitados, assim como o uso de julgadores baseados em modelos difere dos critérios comportamentais definidos no piloto.

Patir et al. (2026) apresentam o DualGauge, que associa testes funcionais e de segurança a tarefas descritas por especificações. Na versão 3, os autores discutem a avaliação das respostas e dos caminhos de execução, utilizando um modelo como oráculo em tempo de execução. Um programa pode produzir a resposta esperada depois de realizar uma operação indevida, o que limita a avaliação baseada apenas na saída. O presente estudo

também combina funcionalidade e segurança, mas usa verificadores de propriedades delimitadas, sem adotar esse julgador semântico.

Quadro 1 - Avaliação dos trabalhos relacionados e relação com o estudo

Estudo	Procedimento de avaliação	Relação e diferença principal
Pearce et al. (2022)	Geração em cenários de segurança e análise dos programas	Evidência de código vulnerável em outra ferramenta e outro conjunto de tarefas
Perry et al. (2023)	Estudo com usuários de assistente de programação	Examina comportamento e confiança humanos, além do código
Khoury et al. (2023)	Geração e solicitações adicionais de segurança	Motiva examinar instruções; envolve interação posterior à geração
Tony et al. (2025)	Técnicas de prompt, inspeção de validade e análise de segurança	Inclui técnicas iterativas e tratamento de respostas inválidas diferente do adotado aqui
Kharma et al. (2026)	Métodos de prompt, análise estática e revisão manual	Compara frequência e composição das fraquezas em outras condições
Basic e Giaretta (2026)	Revisão sistemática de geração, detecção e reparo	Contextualiza a literatura; não é um experimento primário equivalente
Wang et al. (2026)	Testes funcionais e segurança em repositórios Java	Já mede as duas dimensões, com contexto e instrumentos distintos
Patir et al. (2026)	Testes pareados e avaliação de execução com modelo-oráculo	Já avalia especificações e as duas dimensões; difere nos critérios de julgamento

Fonte: elaborado pelo autor a partir dos trabalhos citados (2026).

2.6 Delimitação da contribuição

A literatura já investiga tanto os efeitos dos prompts quanto a relação entre segurança e correção funcional. A contribuição pretendida consiste em comparar quatro condições de especificação em tarefas Python/FastAPI, observando achados estáticos, propriedades comportamentais e aprovação funcional. O detalhamento funcional, a orientação genérica de segurança e os requisitos específicos são examinados por contrastes definidos na metodologia.

As três questões de pesquisa decorrem dessa comparação: a primeira examina o acréscimo de detalhamento funcional; a segunda, as orientações de segurança; e a terceira, o comportamento funcional sob essas condições. N1 fornece uma base comum para N2 e N3, mas os requisitos adicionais também aumentam a extensão dos prompts. O volume de texto permanece uma explicação concorrente para os efeitos observados. A análise considera essa restrição e não atribui à combinação de testes funcionais e de segurança uma originalidade que os trabalhos anteriores já demonstram.

REFERÊNCIAS

BANDIT. Welcome to Bandit. [S.l.], [s.d.]. Disponível em: https://bandit.readthedocs.io/en/latest/. Acesso em: 3 nov. 2026.

BASIC, Enna; GIARETTA, Alberto. From vulnerabilities to remediation: a systematic literature review of LLMs in code security. arXiv:2412.15004, versão 4, 2026. Preprint. Disponível em: https://arxiv.org/abs/2412.15004v4. Acesso em: 3 nov. 2026.

KHARMA, Mohammed et al. An empirical evaluation of LLM-generated code security across prompting methods. arXiv:2605.24298, versão 1, 2026. Preprint. Disponível em: https://arxiv.org/abs/2605.24298v1. Acesso em: 3 nov. 2026.

KHOURY, Raphaël et al. How secure is code generated by ChatGPT? In: IEEE INTERNATIONAL CONFERENCE ON SYSTEMS, MAN, AND CYBERNETICS (SMC), 2023, Oahu. Proceedings [...]. IEEE, 2023. p. 2445-2451. DOI: 10.1109/SMC53992.2023.10394237. Versão dos autores disponível em: https://arxiv.org/abs/2304.09655v2. Acesso em: 3 nov. 2026.

MITRE. About CWE. [S.l.], [s.d.]. Disponível em: https://cwe.mitre.org/about/index.html. Acesso em: 3 nov. 2026.

NASA. SWE-050 - Software Requirements. Software Engineering Handbook, versão D. [S.l.], [s.d.]. Disponível em: https://swehb.nasa.gov/spaces/SWEHBVD/pages/102695421/SWE-050%2B-%2BSoftware%2BRequirements. Acesso em: 3 nov. 2026.

OWASP. Application Security Verification Standard. Versão 5.0.0. [S.l.]: OWASP, 2025. Disponível em: https://owasp.org/projects/asvs. Acesso em: 3 nov. 2026.

PATIR, Rupam et al. DualGauge: automated joint security-functionality benchmarking of specification-only code generation by LLMs and coding agents. arXiv:2511.20709, versão 3, 2026. Preprint. Disponível em: https://arxiv.org/html/2511.20709v3. Acesso em: 3 nov. 2026.

PEARCE, Hammond et al. Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In: IEEE SYMPOSIUM ON SECURITY AND PRIVACY (SP), 2022. Proceedings [...]. IEEE, 2022. p. 754-768. DOI: 10.1109/SP46214.2022.9833571. Versão dos autores disponível em: https://arxiv.org/abs/2108.09293v3. Acesso em: 3 nov. 2026.

PERRY, Neil et al. Do users write more insecure code with AI assistants? In: ACM SIGSAC CONFERENCE ON COMPUTER AND COMMUNICATIONS SECURITY, 2023. Proceedings [...]. New York: ACM, 2023. p. 2785-2799. DOI: 10.1145/3576915.3623157. Versão dos autores disponível em: https://arxiv.org/abs/2211.03622v3. Acesso em: 3 nov. 2026.

SEMGREP. Write rules. [S.l.], [s.d.]. Disponível em: https://docs.semgrep.dev/writing-rules/overview. Acesso em: 3 nov. 2026.

TONY, Catherine et al. Prompting techniques for secure code generation: a systematic investigation. ACM Transactions on Software Engineering and Methodology, v. 34, n. 8, p. 1-53, 2025. DOI: 10.1145/3722108. Versão dos autores disponível em: https://arxiv.org/abs/2407.07064v2. Acesso em: 3 nov. 2026.

WANG, Yanlin et al. RealSec-bench: a benchmark for evaluating secure code generation in real-world repositories. arXiv:2601.22706, versão 1, 2026. Preprint. Disponível em: https://arxiv.org/abs/2601.22706v1. Acesso em: 3 nov. 2026.